
UNIVERSIDAD AUTÓNOMA SAN SEBASTIÁN «UASS»

Facultad de Ciencias, Tecnologías e Innovación

Carrera de Informática · 6.º semestre

TRABAJO PRÁCTICO

Documentación de una API REST con Spring Boot

API Tienda Noticias.lat

Entidad documentada: Producto

Asignatura	Lenguaje de Programación III — Java
Código	26015
Profesor	Lic. Gustavo Lublin
Estudiante	_____
Modalidad	Individual
Valor	15 puntos
Fecha	1 de octubre de 2026

1. Introducción

El proyecto se denomina API Tienda Noticias.lat. La entidad elegida para el alta, la baja, la modificación, el listado y la consulta es Producto. Se trata de cada pieza que la tienda de la redacción ofrece al público: suscripciones, ediciones, libros, objetos y experiencias. La entidad concentra el dato que el resto del sistema necesita para mostrar el catálogo, descontar stock y registrar una venta.

El objetivo de este trabajo es describir la API REST construida con Spring Boot y MongoDB para esa entidad, y dejar registrados los pasos con los que se llegó al resultado. Spring Boot se ocupa de levantar la aplicación y de conectar el código con la base a partir de unas pocas propiedades (Broadcom, 2026). La interfaz de la tienda también existe, pero este documento se concentra en la API de Producto.

El contenido se ordena así: primero las herramientas, la base y los atributos; después los pasos de construcción; luego cada endpoint con el JSON y el código de estado; al final, la conclusión y las referencias.

2. Desarrollo: el proyecto

2.1. Herramientas y versiones

La aplicación se escribió en Java 17, que es la versión indicada para el entorno de la cátedra. El proyecto Maven usa spring-boot-starter-parent 3.5.5. Gutierrez (2024) explica que Spring Boot reduce la configuración inicial gracias a la autoconfiguración y a los starters: cada starter trae una familia de bibliotecas ya compatibles entre sí. En este proyecto se usaron los starters web, data-mongodb, validation y security. El JSON lo produce Jackson, incluido en el starter web. La ejecución se hace con el wrapper de Maven (mvnw), de modo que no hace falta instalar Maven por separado.

2.2. Base de datos

La base es MongoDB, alojada en MongoDB Atlas, en el cluster de nombre nlat. La base de datos de la aplicación se llama tienda_noticias y la colección de la entidad se llama productos. MongoDB guarda cada producto como un documento: una estructura de campos, no una fila de una tabla (MongoDB, Inc., 2026). El identificador lo genera la base y llega al código como una cadena. La dirección de conexión se lee de la variable de entorno MONGODB_URI. La contraseña del usuario de Atlas no se escribe en el repositorio ni en este documento. Solo una dirección IP autorizada en Network Access puede abrir la conexión.

2.3. Atributos de Producto

La clase Producto vive en el paquete lat.noticias.tienda.modelo y se anota con `@Document(collection = "productos")`. Los campos quedan así:

Atributo	Tipo en Java	En el documento	Función
id	String	identificador	Clave del documento. La asigna MongoDB.
sku	String	texto, único	Código corto de la pieza, por ejemplo NT-ANU-2026.
nombre	String	texto	Nombre visible en la tienda y en la API.

slug	String	texto, único	Nombre para la URL. Se arma a partir del nombre.
resumen	String	texto	Texto breve de la ficha.
descripcion	String	texto	Texto largo de la ficha.
precio	BigDecimal	decimal	Precio de venta, con dos decimales.
precioLista	BigDecimal	decimal o nulo	Precio anterior, solo si hay descuento.
stock	int	entero	Unidades disponibles.
categoria	enum Categoria	texto	SUSCRIPCION, EDICION, LIBRO, MERCHANDISING o EXPERIENCIA.
imagen	String	texto	Nombre de la portada, por ejemplo portada-08.
destacado	boolean	booleano	Indica si la pieza aparece en la portada.
activo	boolean	booleano	En false, la pieza quedó dada de baja y no se lista.
creadoEn	LocalDateTime	fecha	Momento del alta.
actualizadoEn	LocalDateTime	fecha	Momento de la última modificación.

El campo activo es la baja lógica. El documento no se borra, porque una venta ya registrada puede citar ese producto. MongoDB no exige declarar este esquema de antemano: la forma del documento la define la aplicación al guardarlo (MongoDB, Inc., 2026).

3. Desarrollo: pasos seguidos

Paso 1. Base de datos

Se creó el proyecto en MongoDB Atlas y se dio de alta un usuario de base con permiso de administración sobre ese proyecto. En Network Access se autorizó la IP desde la que corre la aplicación. La URI sigue la forma `mongodb+srv://usuario:clave@nlat.vi56yw8.mongodb.net/tienda_noticias`. El nombre de la base va en la ruta, antes del signo de pregunta. La clave se carga solo en la variable `MONGODB_URI` de la terminal, no en un archivo versionado.

Paso 2. Proyecto Spring Boot

Se armó un proyecto Maven equivalente al que genera Spring Initializr cuando se eligen Java 17, Maven, Spring Boot 3.5.5 y los starters de web, MongoDB, validación y seguridad. El archivo está en la raíz y se llama `pom.xml`. El fragmento que fija la versión es el siguiente.

Fragmento de pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.5</version>
</parent>
<properties>
  <java.version>17</java.version>
</properties>
```

Paso 3. Configuración de la conexión

La conexión no se escribió en `application.properties`, sino en `src/main/resources/application.yml`. Spring Boot acepta los dos formatos y aplica las mismas claves: `spring.data.mongodb.uri` abre la conexión con MongoDB (Broadcom, 2026). Si la variable de entorno no existe, queda como respaldo una base local.

Fragmento de `application.yml`. La clave real no aparece.

```
spring:
  data:
    mongodb:
      uri: ${MONGODB_URI:mongodb://localhost:27017/tienda_noticias}
      auto-index-creation: true
  server:
    port: 8080
```

Paso 4. Entidad

Se creó la clase `Producto` en `src/main/java/lat/noticias/tienda/modelo/Producto.java`. La anotación `@Document` indica la colección. `@Id` marca el identificador y `@Indexed(unique = true)` impide dos SKU iguales.

Fragmento de `Producto.java`

```
@Document(collection = "productos")
public class Producto {
    @Id
    private String id;

    @Indexed(unique = true)
    private String sku;

    private String nombre;
    private BigDecimal precio;
    private int stock;
    private Categoria categoria;
    private boolean activo;
}
```

Paso 5. Repositorio

Se creó la interfaz `ProductoRepositorio` en el paquete `repositorio`. Extiende `MongoRepository<Producto, String>`, de modo que hereda `guardar`, `buscar por id`, `listar` y `contar`. Spring Data implementa esa interfaz sin que haya que escribir la consulta de cada método (Broadcom, 2026).

Fragmento de `ProductoRepositorio.java`

```
public interface ProductoRepositorio
    extends MongoRepository<Producto, String> {
    Optional<Producto> findBySlug(String slug);
    Optional<Producto> findBySkuIgnoreCase(String sku);
}
```

Paso 6. Servicio

Se creó `ProductoServicio` en el paquete `servicio`. Ahí se valida el SKU repetido, se calcula el slug, se normaliza el precio a dos decimales y se decide si el producto queda activo. El controlador no habla con el repositorio: llama a este servicio. El alta y la modificación pasan por el método `guardar`. La baja pasa por `cambiarActivo(id, false)`.

Fragmento de `ProductoServicio.java`

```
public Producto guardar(ProductoFormulario formulario) {
    // valida portada, precio de lista y SKU único
    producto.setPrecio(CalculadoraVenta.dinero(formulario.getPrecio()));
    producto.setActivo(formulario.isActivo());
    producto.setSlug(slugUnico(producto.getNombre(), producto.getId()));
    return productos.save(producto);
}
```

Paso 7. Controlador

Se creó `ApiProductosControlador` en el paquete `api`, con `@RestController` y `@RequestMapping("/api")`. Cada método publica un verbo HTTP. Las rutas bajo `/api/admin` exigen autenticación Basic del usuario de redacción. Las rutas públicas de consulta no la piden. La seguridad se declaró en `SeguridadConfig`.

Fragmento de `ApiProductosControlador.java`

```
@RestController
@RequestMapping("/api")
public class ApiProductosControlador {

    @GetMapping("/productos")
    public List<ProductoJson> listar() { ... }

    @GetMapping("/productos/{id}")
    public ProductoJson uno(@PathVariable String id) { ... }

    @PostMapping("/admin/productos")
    @ResponseStatus(HttpStatus.CREATED)
    public ProductoJson crear(@Valid @RequestBody ProductoAlta alta) { ... }

    @PutMapping("/admin/productos/{id}")
    public ProductoJson actualizar(@PathVariable String id,
                                   @Valid @RequestBody ProductoAlta alta) { ... }

    @DeleteMapping("/admin/productos/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void eliminar(@PathVariable String id) { ... }
}
```

Paso 8. Ejecución

En la terminal del proyecto se exporta `MONGODB_URI` y se ejecuta `./mvnw.cmd spring-boot:run` en Windows, o `./mvnw spring-boot:run` en Linux y macOS. Cuando el registro muestra Tomcat started on port 8080, la API queda en `http://localhost:8080`. La primera vez, si la colección está vacía, la aplicación carga un catálogo de demostración.

Paso 9. Pruebas

Las pruebas de esta sección se hicieron contra esa API en ejecución. En Postman se elige el verbo, se pega la URL y, en las rutas /api/admin, se activa Authorization de tipo Basic Auth con el usuario de redacción. La clave de esa cuenta está en application.yml, bajo tienda.admin.clave, y no es la contraseña de MongoDB. Un cuerpo JSON lleva la cabecera Content-Type: application/json. Los códigos de estado siguen la semántica de HTTP: 200 es éxito con cuerpo, 201 es un recurso creado, 204 es éxito sin cuerpo y 404 es un recurso que no existe (Fielding y Reschke, 2022).

4. Desarrollo: la API

La entidad Producto se opera con cinco endpoints. El listado y la consulta por id son públicos. El alta, la modificación y la baja piden el usuario de redacción. El cuerpo de entrada usa los nombres nombre, sku, resumen, descripcion, precio, precioLista, stock, categoria, imagen, destacado y activo. La respuesta agrega id, slug y descuento.

4.1. POST /api/admin/productos

Da de alta un producto. El método es POST y la ruta es /api/admin/productos. Requiere autenticación Basic. Si el JSON es válido, responde 201 Created y devuelve el producto ya guardado, con el id que generó MongoDB. Si falta un campo obligatorio, responde 400. Si no se envían las credenciales, responde 401.

Petición de alta

```
POST http://localhost:8080/api/admin/productos
Authorization: Basic (usuario de redacción)
Content-Type: application/json

{
  "nombre": "Póster de cierre",
  "sku": "NT-MER-POS2",
  "resumen": "Una tinta sobre papel obra.",
  "descripcion": "Impreso para la mesa de cierre. Se envía enrollado.",
  "precio": 15.00,
  "stock": 8,
  "categoria": "MERCHANDISING",
  "imagen": "portada-06",
  "destacado": false,
  "activo": true
}
```

Figura 1. Respuesta real del alta. El id lo asigna la base y cambia en cada documento.

```
HTTP/1.1 201 Created

{
  "id": "6abef0eeb4e67d27d79522d3",
  "sku": "NT-MER-POS2",
  "nombre": "Póster de cierre",
  "slug": "poster-de-cierre-2",
  "resumen": "Una tinta sobre papel obra.",
  "descripcion": "Impreso para la mesa de cierre. Se envía enrollado.",
  "precio": 15.00,
  "precioLista": null,
  "descuento": null,
  "stock": 8,
  "categoria": "MERCHANDISING",
  "imagen": "portada-06",
  "destacado": false,
  "activo": true
}
```

Un cuerpo vacío no crea nada. La API responde 400 Bad Request con el detalle por campo:

Figura 2. Alta rechazada por validación. El JSON real incluye también resumen, descripción, imagen y categoría.

```
HTTP/1.1 400 Bad Request

{
  "error": "Datos inválidos",
  "campos": {
    "nombre": "El nombre es obligatorio",
    "sku": "El SKU es obligatorio",
    "precio": "El precio es obligatorio",
    "stock": "El stock es obligatorio"
  }
}
```

4.2. GET /api/productos

Lista los productos activos. El método es GET y la ruta es /api/productos. No pide autenticación. Responde 200 OK con un arreglo JSON. Los productos dados de baja no entran en este listado, porque la consulta filtra activo = true. No se envía cuerpo.

Figura 3. Listado. La respuesta real es un arreglo con todas las piezas activas; aquí se muestra un elemento.

```
GET http://localhost:8080/api/productos
```

```
HTTP/1.1 200 OK
```

```
[
  {
    "id": "6abee3d73d6d36c559bf6351",
    "sku": "NT-ANU-2026",
    "nombre": "Anuario 2026",
    "slug": "anuario-2026",
    "precio": 24.00,
    "precioLista": 29.00,
    "descuento": 17,
    "stock": 58,
    "categoria": "EDICION",
    "imagen": "portada-08",
    "destacado": true,
    "activo": true
  }
]
```

4.3. GET /api/productos/{id}

Consulta un producto por su identificador. El método es GET y la ruta es /api/productos/{id}. No pide autenticación y no lleva cuerpo. Si el id existe, responde 200 OK con un solo objeto. Si no existe, responde 404 Not Found. Este criterio coincide con el uso de 404 cuando el recurso pedido no está en el servidor (Fielding y Reschke, 2022).

Figura 4. Consulta por id existente.

```
GET http://localhost:8080/api/productos/6abee3d73d6d36c559bf6351
```

```
HTTP/1.1 200 OK
```

```
{
  "id": "6abee3d73d6d36c559bf6351",
  "sku": "NT-ANU-2026",
  "nombre": "Anuario 2026",
  "slug": "anuario-2026",
  "resumen": "Las piezas que marcaron el año, editadas para guardar.",
  "precio": 24.00,
  "precioLista": 29.00,
  "descuento": 17,
  "stock": 58,
  "categoria": "EDICION",
  "activo": true
}
```

Figura 5. Consulta por id inexistente.

```
GET http://localhost:8080/api/productos/no-existe
```

```
HTTP/1.1 404 Not Found
```

```
{ "error": "Producto no encontrado" }
```

4.4. PUT /api/admin/productos/{id}

Modifica un producto existente. El método es PUT y la ruta es /api/admin/productos/{id}. Requiere autenticación Basic. El cuerpo reemplaza los datos editables: no se manda el id dentro del JSON, porque ya va en la URL. Si el id existe, responde 200 OK con el producto actualizado. Si el id no existe, responde 404. En la prueba se cambió el precio de 15.00 a 18.00 y se alargó el resumen.

Petición de modificación

```
PUT http://localhost:8080/api/admin/productos/6abef0eeb4e67d27d79522d3
Authorization: Basic (usuario de redacción)
Content-Type: application/json
```

```
{
  "nombre": "Póster de cierre",
  "sku": "NT-MER-POS2",
  "resumen": "Una tinta sobre papel obra, tamaño A3.",
  "descripcion": "Impreso para la mesa de cierre. Se envía enrollado.",
  "precio": 18.00,
  "stock": 8,
  "categoria": "MERCHANDISING",
  "imagen": "portada-06",
  "destacado": false,
  "activo": true
}
```

Figura 6. Respuesta real de la modificación.

```
HTTP/1.1 200 OK
```

```
{
  "id": "6abef0eeb4e67d27d79522d3",
  "sku": "NT-MER-POS2",
  "nombre": "Póster de cierre",
  "slug": "poster-de-cierre-2",
  "resumen": "Una tinta sobre papel obra, tamaño A3.",
  "precio": 18.00,
  "stock": 8,
  "categoria": "MERCHANDISING",
  "activo": true
}
```

4.5. DELETE /api/admin/productos/{id}

Da de baja un producto. El método es DELETE y la ruta es /api/admin/productos/{id}. Requiere autenticación Basic y no lleva cuerpo. Si el id existe, responde 204 No Content: la operación salió bien y no hay cuerpo que devolver (Fielding y Reschke, 2022). Si el id no existe, responde 404 Not Found.

La baja es lógica. El servicio pone activo en false y conserva el documento, para que las ventas viejas sigan nombrando esa pieza. Después del DELETE, GET /api/productos ya no lo lista. GET /api/productos/{id} todavía responde 200, con activo en false, porque el documento sigue en la colección. Así se distingue «no existe» de «existe, pero está de baja».

Figura 7. Baja de un id existente. La respuesta no trae cuerpo.

```
DELETE http://localhost:8080/api/admin/productos/6abef0eeb4e67d27d79522d3
Authorization: Basic (usuario de redacción)

HTTP/1.1 204 No Content
```

Figura 8. Baja de un id que no está en la colección.

```
DELETE http://localhost:8080/api/admin/productos/no-existe

HTTP/1.1 404 Not Found

{ "error": "Producto no encontrado" }
```

Figura 9. El mismo id, después de la baja. Sigue guardado y queda fuera del listado público.

```
GET http://localhost:8080/api/productos/6abef0eeb4e67d27d79522d3

HTTP/1.1 200 OK

{
  "id": "6abef0eeb4e67d27d79522d3",
  "nombre": "Póster de cierre",
  "precio": 18.00,
  "activo": false
}
```

5. Conclusión

Se obtuvo una API REST capaz de crear, listar, consultar, modificar y dar de baja productos de la tienda, con los códigos 201, 200, 204 y 404 según el caso. La misma entidad alimenta el catálogo visible en <http://localhost:8080> y el panel de redacción. La conexión a MongoDB Atlas quedó fuera del código, en una variable de entorno.

Una dificultad fue la versión de Java: el primer pom.xml pedía el release 21 y el JDK instalado para la cátedra es el 17, así que la compilación se detenía con el mensaje `release version 21 not supported`. Se resolvió fijando `java.version` en 17 dentro de pom.xml, sin cambiar el lenguaje ni las bibliotecas. Otra dificultad fue el acceso a Atlas: la base está en internet y solo acepta las IP cargadas en Network Access; además, la URI tiene que nombrar la base `tienda_noticias`. Se resolvió autorizando la IP y pasando `MONGODB_URI` al arrancar, sin guardar la contraseña en el proyecto.

También hubo que decidir qué significa borrar. Quitar el documento impediría reconstruir una venta histórica. Se resolvió con la baja lógica sobre el campo `activo`, y el endpoint `DELETE` quedó asociado al código 204. El aprendizaje principal es que una API pequeña se sostiene en tres piezas separadas: la entidad que describe el dato, el repositorio que lo persiste y el controlador que traduce el HTTP. Spring Boot une esas piezas al leer la URI de MongoDB (Broadcom, 2026).

6. Referencias

Broadcom. (2026). MongoDB. En Spring Boot reference documentation (versión 3.5).
<https://docs.spring.io/spring-boot/reference/data/nosql.html>

- Fielding, R. y Reschke, J. (Eds.). (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force.
<https://www.rfc-editor.org/rfc/rfc9110>
- Gutierrez, F. (2024). Pro Spring Boot 3: An authoritative guide with best practices (3.^a ed.). Apress.
<https://doi.org/10.1007/978-1-4842-9294-5>
- MongoDB, Inc. (2026). Documents. En MongoDB manual.
<https://www.mongodb.com/docs/manual/core/document/>